Operator Overloading

Binary operator (+, -, *, /, %)

- Using + operator

```
#include<iostream>
using namespace std;
class A{
    int x,y;
    public:
    A(){x=0;y=0;}
    A(int i,int j){x=i;y=j;}
    A operator+(A obj){
        A temp;
        temp.x=x+obj.x;
        temp.y=y+obj.y;
        return temp;
    }
    void show(){
        cout<<x<<" "<<y<<endl;
    }
};
int main(){
    A obj1(10,20),obj2(30,40),obj3;
    obj3 = obj1+obj2;
    obj3.show();
}
```

NB: to another operator just change the sign

Unary Operator [+, -, ++, - -]

- Using ++ operator

```
#include<iostream>
using namespace std;
class A{
    int x,y;
    public:
    A(){x=0;y=0;}
    A(int i,int j){x=i;y=j;}
    A operator++(){
        x++;
        y++;
    }
    void show(){
        cout<<x<<" "<<y<<endl;
    }
};
int main(){
    A obj1(10,20);
    ++obj1;
    obj1.show();

}
```

NB: to another operator just change the sign

- Using (-) operator as unary

```
#include<iostream>
using namespace std;
class A{
    int x;
    public:
    A(){x=0;}
    A(int i){x=i;}
    A operator-(){
        return A(-x);
```

```
        }
        void show(){
            cout<<x<<endl;
        }
    };
    int main(){
        A obj1(10),obj;
        obj = -obj1;
        obj.show();

    }
```

Assignment Operator [=, +=, *=, /=, -=, %=]

- Using (+=) operator

```
#include<iostream>
using namespace std;

class A{
    int x;
    public:
    A(){x=0;}
    A(int i){x=i;}
    A operator+=(A obj){
        this->x+=obj.x;
        return *this;
    }
    void show(){
        cout<<x<<endl;
    }
};

int main(){
    A obj1(10),obj2(20);
    obj1+=obj2;
    obj1.show();

}
```

NB: to another operator just change the sign

Relational Operator [>, < , = =, <=, >=]

```
#include<iostream>
using namespace std;

class A{
    int x;
    public:
    A(){x=0;}
    A(int i){x=i;}
    bool operator>(A obj){
        return x>obj.x;
    }
    void show(){
        cout<<x<<endl;
    }
};
int main(){
    A obj1(10),obj2(20);
   if(obj1>obj2){
    cout<<"Obj1 Greater";
   }else{
    cout<<"Obj2 Greter";
   }

}
```

NB: to another operator just change the sign

Logical Operator [&, | |, !]

```
class A{
    int x,y;
    public:
    A(){x=0;y=0;}
    A(int i,int j){x=i;y=j;}

    int operator&&(A obj){
        return (x && obj.x) && (y && obj.y);
    }

    bool operator==(A obj){
        return x == obj.x && y == obj.y;
    }
    bool operator|(A &obj){
        return (x && obj.x) || (y && obj.y);
    }
};

int main(){

    A obj1(10,10),obj2(10,0),obj3(5,3),obj4(6,2);

    if(obj1==obj2){
        cout<<"SAME"<<endl;
    }else{
        cout<<"Not Same"<<endl;
        }

    if(obj2 | obj3){
        cout<<"obj1 obj2 is true"<<endl;
    }else{
        cout<<"false"<<endl;
    }
    if(obj2 && obj3){
        cout<<"obj1 obj2 is true"<<endl;
    }else{
        cout<<"false"<<endl;
    }
```

Assignment Operator (=) Overloading

```
#include<iostream>
using namespace std;
class Distance {
private:
int feet;
int inches;
public:
Distance() {feet = 0; inches = 0;}
Dstance(int f, int i) {feet = f; inches = i;}
Distance operator = (const Distance &D ) {
feet = D.feet;
inches = D.inches;
}//
method to display distance
Distance displayDistance() {
cout << "F: " << feet
cout << " I:" << inches << endl;
}}
;
int main() {
Distance D1(11, 10), D2(509, 119);
cout << "First Distance : ";
D1.displayDistance();
cout << "Second Distance :";
D2.displayDistance();
// use assignment operator
D1 = D2;
cout << "First Distance :";
D1.displayDistance();
return 0;
}
```

Operator Overloading in C++ Using Friend Function

```
#include<bits/stdc++.h>
using namespace std;

class A{
    int x,y;

    public:
    A(){x=0;y=0;}
    A(int i,int j){x=i,y=j;}

    // int operator!=(A obj1){                        //relational
    //     return x!=obj1.x and y!=obj1.y;
    // }
    friend int operator!=(A &obj1,A &obj2);           //relational using friend funtion

    A operator++(int){                                //unary
        x++;
        y++;
        return *this;
    }

    A operator+(A obj){           //binary
        A temp;
        temp.x = x+obj.x;
        temp.y = y+obj.y;
        return temp;
    }

    friend A operator+(int i,A &obj2);               //to solve int+object problem by using friend funtion
    friend A operator+=(A &obj1,A &obj2);           //assighnment operator using friend function
    friend int operator&&(A &obj1,A &obj2);         //logical operator using friend function

void show(){
        cout<<x<<" "<<y<<endl;
    }
};

int operator!=(A &obj1,A &obj2){
    return obj1.x!=obj2.x and obj1.y!=obj2.y;
}
A operator+=(A &obj1,A &obj2){
    obj1.x+=obj2.x;
    obj1.y+=obj2.y;
    return obj1;

}
A operator+(int i,A &obj2){
    A temp;
    temp.x = i+obj2.x;
    temp.y = i+obj2.y;
    return temp;
}
int operator&&(A &obj1,A &obj2){
    return (obj1.x && obj2.x)  && (obj1.y && obj2.y);
}
 int main(){
    A obj1(10,30),obj2(30,0),obj3;
    if(obj1 && obj2){
        obj3 = 20+obj2;
        obj3.show();
    }else{
        cout<<"+="<<endl;
       obj1+=obj2;
       obj1.show();
    }
 }
```

Inheritance

Types of Inheritance

```
#include<iostream>
using namespace std;
                              >>>>>>>>>>>Singal Inheritance<<<<<<<<<<<

class A{
    public:
    void f1(){
        cout<<"Class A";
    }
};
class B:public A{
     public:
    void f2(){
        cout<<"Class B";
    }
};
int main(){
    B obj;
    obj.f1();
}

                                    >>>>>>>>Multiple Inheritance<<<<<
class A{                          //>>Base Class
    public:
    void f1(){
        cout<<"Class A"<<endl;
    }
};
class B{                        //>>Base Class
    public:
    void f2(){
        cout<<"Class B"<<endl;
    }
};
class C:public A,public B{    //>>Derived Class
    public:
    void f3(){
        cout<<"Class C"<<endl;
    }
};
int main(){
    C obj;
    obj.f1();
    obj.f2();
}

                                  >>>>>> Hierarchical Inheritance<<<<<
There are Multiple derived class and one base class

class A{                             //>>Base Class
    public:
    void f1(){
        cout<<"Class A"<<endl;
    }
};
class B:public A{                       //>>Derived Class
    public:
    void f2(){
        cout<<"Class B"<<endl;
    }
};
class C:public A{                       //>>Derived Class
    public:
    void f3(){
```

```
        cout<<"Class C"<<endl;
    }
};
int main(){
    C obj;
    B obj2;
    obj.f1();
    obj2.f1();
}
```

>>>>Multilevel Inheritance<<<<<

One class is derived from a class which also derived from another class

```
class A{                            //>>Base Class
    public:
    void f1(){
        cout<<"Class A"<<endl;
    }
};

class B:public A{                       //>>Derived from Class A and Base class of Class C
    public:
    void f2(){
        cout<<"Class B"<<endl;
    }
};
class C:public B{                       //>>Derived Class

    public:
    void f3(){
        cout<<"Class C"<<endl;
    }
};

int main(){
    C obj;
    obj.f1();
}
```

>>>>Hybrid Inheritance<<<<<

```
class A{                                            //>>Base Class
    public:
    void f1(){
        cout<<"Class A"<<endl;
    }
};
class B: virtual public A{                          //>>Derived from Class A and Base class of Class C
    public:
    void f2(){
        cout<<"Class B"<<endl;
    }
};
class C: virtual public A{                          //>>Derived Class
    public:
    void f3(){
        cout<<"Class C"<<endl;
    }
};
class D:public B,C{
};
int main(){
    D obj;
    obj.f1();
    obj.f2();


}
```

Diamond Problem Solution

```
class A{
    int x;
    public:
    void ba(){
        cout<<"In Base Class";
    }
};
class B:virtual public A{
    public:
    void db(){
        cout<<"Derived Class B";
    }
};

class C:virtual public A{};
class D:public B,public C{};

int main(){
   D obj;
   obj.ba();
   obj.db();
}
```

Solving Ambiguity in Multiple Inheritance

```
class A{
    public:
    void show(){
        cout<<"In Base Class";
    }
};
class B{
    public:
    void show(){
        cout<<"Derived Class B";
    }
};
class C:public A,public B{};
int main(){
   C obj;
   obj.A::show();
   obj.B::show();
}
```

Invoke Base class parameterized constructor inside derived class parameterized constructor

```
class A{
    int x;
    public:
    A(int i){x=i;}
    void show(){
        cout<<"BASE VALUE: "<<x;
    }
};
class B:public A{
    int y;
    public:
    B(int j,int k):A(k){
        y=j;
    }
    void show(){
        cout<<"Derived VALUE: "<<y;
    }
};
int main(){
  B obj(10,5);
  obj.B::show();
```

```
}
```

Nesting Of Classes

```
class A{
    public:
    class B{
        int x;
        public:
        B(int i){x=i;}
        void show(){
            cout<<x<<endl;
        }

    };
};
int main(){
  A::B obj(10);
  obj.show();
  return 0;

}
```

SOME PROBLEM Of Inheritance

1. **Create a base class called vehicle that stores the number of wheels a vehicle has and the range of vehicle. Create a derived class called car that inherits vehicle and also store the number of passengers. Then, create a derived class called truck that inherits vehicle and also stores the information of load limit, Display the information**

```
class vehicle{
    int wheel,range;
    public:
    vehicle(int i,int j){wheel = i;range =j;}
    void showv(){
        cout<<"wheel"<<" "<<wheel<<endl;
        cout<<"range"<<" "<<range<<endl;
    }
};
class car: public vehicle{
    int passenger;
    public:
     car(int i,int j,int k):vehicle(j,k){passenger = i;}
    void showcar(){
        cout<<"Passengr"<<" "<<passenger<<endl;
    }
};
class truck: public vehicle{
    int load;
    public:
    truck(int i,int j,int k):vehicle(j,k){load = i;}
    void showtruck(){
        cout<<"Load"<<" "<<load<<endl;
    }
};
int main(){
    int x,y,z,i,j,k;
    cin>>x>>y>>z;
    cin>>i>>j>>k;
    truck truckobj(x,y,z);
    car carobj(i,j,k);
    truckobj.showtruck();
    truckobj.showv();
    carobj.showcar();
    carobj.showv();

}
```

2. **Create a class hierarchy for a bookstore's inventory system with a base class Item (title and Price) and two subclasses, Book (author, genre, ISBN) and CD (artist, genre, duration). Subclass further with Fiction and Non-Fiction for books and Rock and Pop for CDs, adding category-specific data. Demonstrate the use of this hierarchy by creating and displaying sample instances of various books and CDs, showcasing their detaiIs.**

```
class item{
    protected:
    string title;
    int price;
    public:
    item(string i,int j){title = i;price =j;};
    virtual void display()=0;
};
class Books:public item{
    protected:
    string author, genre;
    int isbn;
    public:
    Books(string i,string j,string  k,int l,int m):item(i,m){
        author = j; genre = k;
        isbn = l;
    }
    virtual void display(){
        cout << "Title: " << title << "\nPrice: " << price << "\nAuthor: " << author << "\nGenre: "
<< genre << "\nISBN: " << isbn << endl;
    }
};
class Cd:public item{
    string artist,genre;
    int duration;
    public:
     Cd(string i,string j,string  k,int l,int m):item(i,m){
        artist = j; genre = k;
        duration = l;
    }
    virtual void display(){
cout << "Title: " << title << "\nPrice: " << price << "\nArtist: " << artist << "\nGenre: " << genre
<< "\nDuration: " << duration << " mins" << endl;
    }
};
class Fiction:public Books{
    string author, genre;
    int isbn;
    public:
    Fiction(string i,string j,string  k,int l,int m):Books(i,j,k,l,m){
        author = j; genre = k;
        isbn = l;
    }

};
class Nonficton:public Books{
    public:
    Nonficton(string i,string j,string  k,int l,int m):Books(i,j,k,l,m){}
};

class Rock:public Cd{
    public:
      Rock(string i,string j,string  k,int l,int m):Cd(i,j,k,l,m){}
};
class Pop:public Cd{
     public:
     Pop(string i,string j,string  k,int l,int m):Cd(i,j,k,l,m){}
};
int main(){
    Fiction fictionBook("The Great Gatsby", "F. Scott Fitzgerald", "Novel", 1234567890,10.99);
    Nonficton nonFictionBook("Sapiens", "Yuval Noah Harari", "History", 7654321,15.99);
    Rock rockCD("Rock Anthems", "Various Artists", "Rock", 1209,99);
```

```
    Pop popCD("Pop Hits", "Various Artists", "Pop", 90,8.99);
    fictionBook.display();
    nonFictionBook.display();
    rockCD.display();
    popCD.display();

}
```

Virtual Function

Program that shows the uses of virtual function.

```
Include<iostream>
using namespace std;
class base{
    int x;
    public:
    virtual void show(){
        cout<<"base class"<<endl;
        cout<<x<<endl;
    }
    virtual void set(int i){
        x=i;
    }

};

class derived1:public base{
    int x;
    public:
    void show(){
        cout<<"derived1"<<endl;
        cout<<x<<endl;
    }
    void set(int i){
        x=i;
    }
};

class derived2:public base{
    int x;
    public:
    void set(int i){
        x=i;
    }
    void show(){
        cout<<"Derived2"<<endl;
        cout<<x<<endl;
    }
};
int main(){
   base *p;
   base b_obj;
   derived1 d1_obj;
   derived2 d2_obj;


   p = &b_obj;
   p->set(10);
   p->show();

   p = &d1_obj;
   p->set(20);
   p->show();

   p = &d2_obj;
   p->set(30);
   p->show();

return 0;

}
```

Template

Function Templates or Generic functions

```
#include<bits/stdc++.h>
using namespace std;

template <typename T>

T sum(T i,T j){
    return i+j;
}

int main(){
    int a=10,b=20;
    float c=9.2,d=3.5;
    double e=900.23,f=300.56;
    long long g=1232,h=2023;
    cout<<"Sum of int value: "<<sum(a,b)<<endl;
    cout<<"Sum of float value: "<<sum(c,d)<<endl;
    cout<<"Sum of double value: "<<sum(e,f)<<endl;
    cout<<"Sum of long ling value: "<<sum(g,h)<<endl;
}
```

Class Templates or Generic classes

```
#include<bits/stdc++.h>
using namespace std;
template <class T>

class calculate{
    T x,y;
    public:
    calculate(T i,T j){
        x=i;
        y=j;
    }
    T add(){return x+y;}
    T minus(){return x-y;}
    T multiply(){return x*y;}
    T devide(){return x/y;}

    void display(){
        cout<<add()<<endl;
        cout<<minus()<<endl;
        cout<<multiply()<<endl;
        cout<<devide()<<endl;
    }
};
int main(){
    calculate <int>intobj(30, 20);
    calculate <float>floatobj(30.2, 20.33);
    cout<<"int data type:"<<endl;
    intobj.display();
    cout<<"float data type:"<<endl;
    floatobj.display();
}
```

I/O FILE

Format flag IO

```
    cout.width(10);
    cout.setf(ios::left);
    cout.fill('*');
    cout<<"Hello"<<endl;

    cout.setf(ios::right);
    cout.width(10);
    cout.fill('*');
    cout<<"Hello"<<endl;

    cout.setf(ios::showpos);
    cout<<100<<endl;

    cout.setf(ios::showpoint);
    cout<<100.3<<endl;

    cout.unsetf(ios::dec);
    cout.setf(ios::hex);
    cout<<100<<endl;

    cout.setf(ios::boolalpha);
    cout<<true;
```

Own Manipulator

```
ostream &display(ostream &stream){
    stream.width(10);
    stream.precision(10);
    stream.fill('#');
    stream.setf(ios::left);
    stream.setf(ios::showpos);
    return stream;
}
int main(){
    cout<<display<<100;
}
```

Creating Extractor and Inserter

```
class extract{
    int x,y;
    public:
    extract(){x=0;y=0;}
    extract(int i,int j){x=i;y=j;}
    friend ostream &operator<<(ostream &stream,extract obj);     //inserter
    friend istream &operator>>(istream &stream,extract &obj);    //extractor
};
ostream &operator<<(ostream &stream,extract obj){
    stream<<obj.x<<" "<<obj.y<<endl;
    return stream;
}
istream &operator>>(istream &stream,extract &obj){
    cout<<"ENTER THE VALUE: ";
    stream>>obj.x>>obj.y;
    return stream;
}
int main(){
    extract obj1;
    cin>>obj1;
    cout<<obj1;
}
```

Creating File

```
#include<bits/stdc++.h>
using namespace std;

int main(){
    string name,sem,code,title;
    ofstream outfile;                   //openfile in write mode
    outfile.open("NEWFILE.txt");


    getline(cin,name);
    outfile<<"Name: "<<name<<endl;

    getline(cin,sem);
    outfile<<"Semester: "<<sem<<endl;

    getline(cin,code);
    outfile<<"Course: "<<code<<endl;

    getline(cin,title);
    outfile<<"Course Title: "<<title<<endl;


    outfile.close();                        //close file

    ifstream infile;                        //open file in read mode
    infile.open("NEFILE.txt");

    getline(infile,name);


    cout<<name;

    return 0;
}
```